FastXeBench: A Reproducible Benchmark and Fidelity Study of Edit-to-PDF Latency for XeLaTeX on Windows

Zirui Xu
Nanjing Normal University
19230444@njnu.edu.cn

Abstract

FastXeBench is a reproducible benchmark study of edit-to-PDF latency for XeLaTeX on Windows. The harness uses a prime-plus-measure protocol so that incremental scenarios time only the post-edit build rather than a fresh build. Our formal core matrix covers four workloads, five scenarios, and three main configurations, producing 45 workload-scenario-config groups and 900 recorded runs. We report per-group medians, IQRs, and bootstrap 95% confidence intervals, with corrected nonparametric comparisons deferred to the appendix. The headline result is workload-sensitive rather than uniform: `latexmk` reliably improves steady incremental edits, while `minted` cache is the strongest safe optimization on code-heavy documents, reducing incremental latency on `W5` from about 43 seconds to about 6 seconds for a 7.16x speedup under full visual equality. All retained main-matrix runs preserve visual equality with the baseline. Two real-project-derived validation workloads reproduce the `latexmk` pattern, and a scoped Windows storage follow-up shows that the observed `C:` profile root is about 1.20x slower than the repository-local `D:` root. These results position FastXeBench as both a benchmark artifact and a practical guide for Windows-local XeLaTeX workflows, and the public repository exposes the harness together with the paper-regeneration path.

1 Introduction

XeLaTeX users often optimize their local edit-compile loop without a reproducible method for comparing toolchain choices. In editor setups that compile on save and refresh PDF previews automatically, even a few extra seconds per edit turn directly into visible waiting time and interrupted flow. Recent evidence that TeX outputs can vary across engines and distribution versions means that latency alone is not a sufficient optimization target; fidelity must be measured alongside speed [14]. FastXeBench targets that gap with a runnable benchmark harness centered on edit-to-PDF latency rather than fresh-build latency.

This paper makes four concrete contributions. First, it packages a Windows-local XeLaTeX benchmark artifact with explicit workload, scenario, and configuration boundaries. Second, it fixes the measurement target at edit-to-PDF latency through a prime-plus-measure protocol rather than a fresh-build proxy. Third, it treats output fidelity as a release gate by combining PDF hashes with page-level raster comparison. Fourth, it turns the resulting dataset into practical guidance: `latexmk` is a steady incremental default, `minted` cache is the strongest safe code-heavy optimization in the present matrix, and excluded configurations are kept explicit rather than folded into optimistic recommendations.

2 Related Work and Positioning

FastXeBench sits at the intersection of TeX-output reproducibility, rigorous benchmarking, and developer feedback latency. On the document side, Tan and Rigger show that TeX-produced

PDFs can diverge substantially across engines and versions, which motivates treating output fidelity as a first-class constraint rather than assuming that a faster build is automatically acceptable [14]. On the measurement side, rigorous benchmarking work emphasizes repeatability, stable sampling conditions, and cautious interpretation of small timing deltas [9, 11]. These concerns carry over directly to edit-loop measurements, where build latency is often dominated by caching, reruns, and filesystem behavior rather than by a single deterministic kernel.

The broader systems context comes from build and feedback research. Build systems research formalizes incremental dependency tracking and the trade-offs between soundness, reuse, and rebuild scope [13, 7]. In parallel, developer-productivity work treats latency and predictability as central components of useful feedback loops rather than as secondary runtime trivia [1, 8, 10]. FastXeBench adopts that framing, but narrows it to a concrete measurement target that is common in TeX authoring and under-specified in prior tooling discussions: edit-to-PDF latency on a Windows-local XeLaTeX workflow.

Tool-specific prior art still matters because the benchmark matrix is intentionally grounded in real TeX choices rather than in abstract scheduler variants. `latexmk` automates reruns and dependency tracking [3], `minted` provides a realistic cache-heavy code path [4], `mylatexformat` represents preamble dumping [5], PGF/TikZ covers figure-heavy edit loops [6], and Tectonic provides a modern cached-engine comparison point [15]. The difference is scope: FastXeBench turns those tool paths into a bounded Windows-local benchmark artifact with explicit workload definitions, scenario-level mutations, and a fidelity gate that decides which optimizations remain recommendation candidates. Prior work motivates fidelity-aware measurement and rigorous benchmarking, but it does not package a Windows-local XeLaTeX benchmark artifact around scenario-level edit mutations and a release-gated visual-equality check.

3 Benchmark Design

Protocol v0.2 uses prime-plus-measure semantics for incremental scenarios: a stable state is built in an isolated run directory, a controlled edit is applied, and only the second build is timed. The formal core matrix covers four workloads, five scenarios, and three main-matrix configurations (`B0`, `B1`, and `B4`). The core workloads intentionally span text-and-math, CJK/native-font, TikZ-heavy, and code-heavy documents; the second-batch datasets then add one explicit `fontspec`-based validation workload [2], one PGFPlots-based validation workload, and one scoped Windows storage comparison.

All formal measurements use the single Windows-local host summarized in Table 1. Table 2 makes the workload objects concrete rather than treating `W1`–`W5` as opaque labels, and Table 3 fixes the operational definition of each scenario-level mutation. Each recorded run executes in an isolated directory under the dataset root. Scenario `S0` measures a clean build directly, while `S1`–`S4` first build a stable state, apply one controlled mutation, and then time only the post-edit rebuild. Reference PDFs are generated per workload/scenario pair and reused for fidelity comparisons.

We intentionally lock the formal matrix to Windows 11 because the platform is not incidental noise in this study. XeLaTeX compilation touches many small files, auxiliary reruns, and local fonts, so NTFS metadata behavior, background scanning, and Windows-local storage layout are part of the empirical question rather than nuisance variables to be abstracted away [12]. The scoped storage follow-up is therefore presented as part of the benchmark story, not as an unrelated afterthought.

Configurations under study. The main matrix keeps only configurations that remain both interpretable and visually safe in the present environment: direct XeLaTeX (`B0`), `latexmk` automation (`B1`) [3], and `minted` cache reuse on the code-heavy workload (`B4`) [4]. The excluded configurations remain visible but are not folded back into the headline recommendations: `B2` is

Table 1: Experimental setup for the formal core dataset.

Item	Value
Host CPU	12th Gen Intel(R) Core(TM) i7-12700H (14 cores / 20 threads)
Memory	15.6 GiB physical RAM
OS	Microsoft Windows 11 Home (Chinese edition), version 10.0.26100 (build 26100)
Machine	ASUS TUF Gaming F15 FX507ZV4_FX507ZV4
Power plan	Balanced
Repository volume	D: NTFS fixed volume; Defender performance mode status 0
Primary TeX engine	XeTeX 3.141592653-2.6-0.999997 (TeX Live 2025)
Automation	Latexmk, John Collins, 15 June 2025. Version 4.87; hyperfine 1.20.0
Follow-up engine	tectonic 0.15.0
PDF comparison	pdftoppm version 25.02.0 at 144 dpi plus per-page PNG SHA-256
Font path	`ctexart` on the local Windows CJK path; R1 validation uses TeX Gyre Pagella, Heros, and Cursor.
Formal protocol	`v0.2-prime-measure`; 4 workloads, 5 scenarios, 3 main configurations
Runs per group	3 warmups and 20 recorded runs for the formal core matrix

Table 2: Workload anatomy for the four main-matrix workloads.

Workload	Pages	Lines	Bib	TikZ	CJK	Native-font	Minted	Expected bottleneck
W1 Text+Math	2	70	2	1	no	no	no	Auxiliary reruns plus one lightweight TikZ figure.
W2 CJK+Font	2	69	2	1	yes	yes	no	Windows-local CJK and native-font typesetting path.
W4 TikZ	3	76	2	3	no	no	no	TikZ/pgfplots rendering and figure-page regeneration.
W5 Minted	2	82	2	0	no	no	yes	Pygments highlighting, minted cache reuse, and auxiliary reruns.

a XeLaTeX+`mylatexformat` compatibility-limited negative result [5], `B3` is an appendix-only TikZ externalization path because it reproduced visual differences on `W4` [6], `B6` is retained only as a development-oriented Tectonic reference because its diffs persisted in triage [15], and `B7` is a scoped Windows storage follow-up rather than a main-matrix build configuration.

Fidelity is checked in two stages. The harness first records a whole-file SHA-256 for the candidate PDF and its workload/scenario reference. It then rasterizes both PDFs with `pdftoppm -png -r 144` and compares per-page PNG hashes. If any page hash differs, the run is marked `visual_equal = false` and the changed page indices are recorded together with diagnostic diff images. In the present paper, `binary_equal` is retained as a secondary indicator only; the main fidelity gate is `visual_equal`.

Primary statistical summaries use group medians, IQRs, and 95% bootstrap confidence intervals computed from the recorded wall-clock latencies. Each formal group uses three warmups before 20 recorded runs so that cold-start effects, filesystem preheating, and early cache population do not dominate the measurements. Appendix Table 7 adds Holm-corrected two-sided Wilcoxon signed-rank tests for the non-baseline configurations. Because the current artifact records repeated runs rather than an explicit cross-configuration pairing ID, these appendix tests pair runs by within-group recorded order after sorting `run_id`; we therefore treat them as supporting evidence rather than stronger than the descriptive summaries themselves.

Table 3: Operational scenario definitions used by protocol `v0.2-prime-measure`.

Scenario	Prime?	Mutation	Edit size	Expected rebuild scope
S0 clean	No	No source mutation; direct clean build.	N/A	Whole document from scratch.
S1 text	Yes	`\benchtexttoken`: `ALPHA` → `BETA`	Single body-token replacement	Body text update with possible local page reflow; no bibliography or preamble invalidation.
S2 bib	Yes	`\cite{refalpha}` → `\cite{refalpha,refbeta}`	Single cite expansion	Triggers `.aux`/`.bbl` updates and a bibliography rerun.
S3 figure	Yes	`\benchfiguretoken`: `1.00` → `1.25`	Single numeric figure parameter	Forces figure/TikZ regeneration without changing bibliography or preamble state.
S4 preamble	Yes	`\benchpreambletoken`: `P0` → `P1`	Single preamble-token replacement	Invalidates the preamble and forces the document back through the XeLaTeX path.

Table 4: Baseline latencies for `B0_baseline_xelatex`. Incremental latency is the median over S1, S3, and S4.

Workload	Clean (s)	Bib (s)	Incremental (s)
W1 Text+Math	7.06	7.07	7.07
W2 CJK+Font	7.07	7.07	7.07
W4 TikZ	10.08	10.07	10.07
W5 Minted	43.06	43.07	43.07

4 Results

The formal core dataset contains 45 workload-scenario-config groups and 900 recorded runs. All recorded runs succeeded, and every main-matrix row currently preserves visual equality with the baseline. This means the primary conclusions are driven by steady-state latency rather than speed-versus-fidelity trade-offs inside the core matrix. Table 5 compresses the main matrix, while Appendix Table 7 reports the per-group medians, IQRs, bootstrap confidence intervals, and corrected nonparametric tests.

Table 4 makes the XeLaTeX baseline latencies explicit, because speedup is not a substitute for absolute delay. Figure 1 then shows that the main result is workload-sensitive rather than uniform. For the W1 text+math and W2 CJK+font workloads, `latexmk` reduces the S1, S3, and S4 edit latencies from about 7.1 seconds to about 4.0 seconds, yielding roughly 1.76x speedups, but it is slower than the baseline on clean builds and bibliography edits. For W4 TikZ, `latexmk` is close to neutral on clean builds and bibliography edits, while reducing text, figure, and preamble edits from about 10.1 seconds to about 4.0 seconds, for about 2.50x speedups. For W5 minted, `latexmk` lowers clean builds from about 43.1 seconds to about 38.0 seconds, and lowers S1, S3, and S4 to about 14.0 seconds, for about 3.07x speedups.

`minted` cache is the strongest stable optimization in the current core matrix. On W5 minted, it reduces clean-build latency to about 19.0 seconds (2.26x over baseline), reduces bibliography edits to about 16.0 seconds (2.69x), and reduces text, figure, and preamble edits to about 6.0 seconds (about 7.16x), while preserving visual equality in all recorded runs. Table 5 also makes the engineering meaning of those speedups explicit: the incremental median saving is about 3 seconds on W1/W2, about 6 seconds on W4, about 29 seconds for `B1` on W5, and about 37

Table 5: Formal core rollup for non-baseline configurations. Incremental latency is the median over S1, S3, and S4, and saved seconds are measured against the same-workload XeLaTeX baseline.

Workload	Config	Clean (s)	Clean ×	Bib (s)	Bib ×	Incremental (s)	Incremental ×	Inc. saved (s)
W1 Text+Math	B1 latexmk	8.02	0.88	8.02	0.88	4.02	1.76	3.05
W2 CJK+Font	B1 latexmk	9.02	0.78	9.02	0.78	4.02	1.76	3.05
W4 TikZ	B1 latexmk	10.02	1.01	10.02	1.01	4.02	2.51	6.05
W5 Minted	B1 latexmk	38.02	1.13	38.01	1.13	14.01	3.07	29.05
W5 Minted	B4 minted cache	19.02	2.26	16.02	2.69	6.01	7.16	37.05

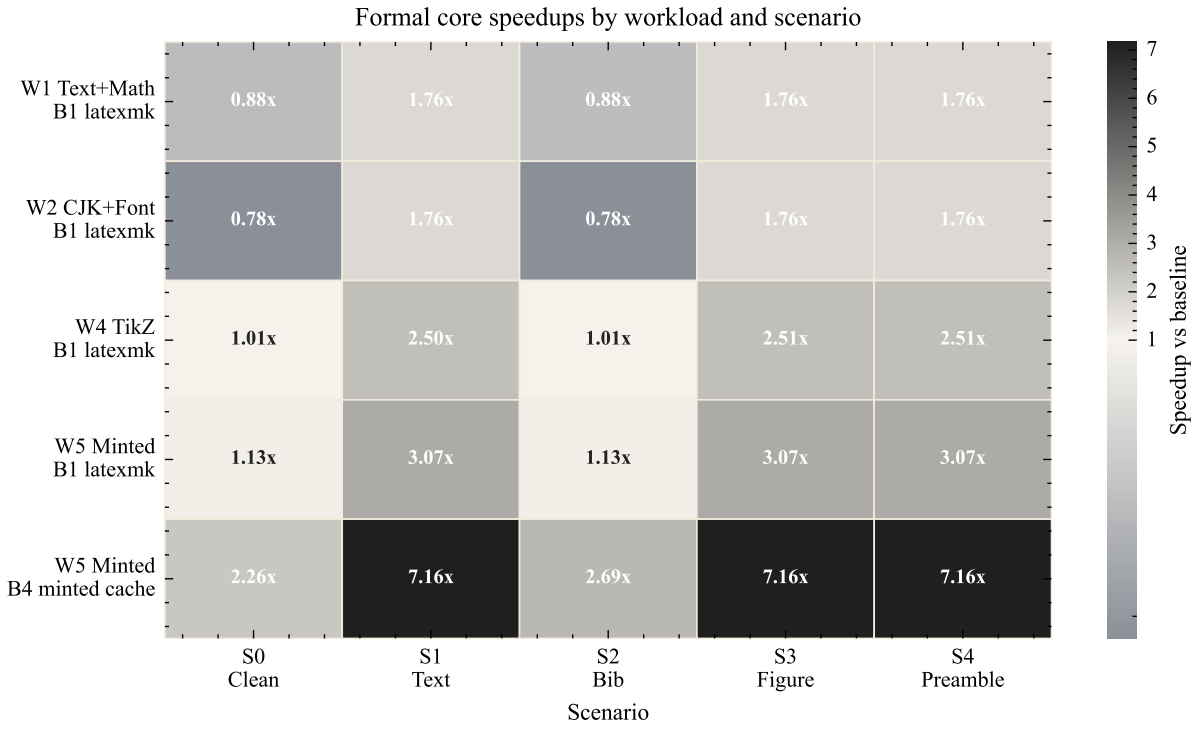

Figure 1: Formal speedup matrix for the non-baseline configurations. Values above 1.0 indicate lower latency than the XeLaTeX baseline, and each cell is numerically annotated so the plot remains interpretable in grayscale print.

seconds for `B4` on W5. Figure 2 makes this separation visible at the run level rather than only at the level of group medians.

Appendix Table 7 shows that these group differences remain statistically distinguishable after Holm correction. The headline positive cells in the paper, namely the `B1` incremental cases and all retained `B4` cases, remain strongly significant with corrected $p < 0.001$. That result should still be read carefully: corrected significance does not imply universal speedup, because several `B1` clean-build and bibliography cells are significantly slower than the direct XeLaTeX baseline even while the incremental cells remain significantly faster.

The narrow IQRs and bootstrap intervals in Appendix Table 7 are a consequence of the protocol rather than of aggressive rounding. All runs execute on one host, under one power plan, in isolated directories, after explicit warmups, and the recorded wall-clock values are kept at millisecond resolution before being summarized to two decimals in the paper tables. The artifact is therefore optimized for stable within-host comparison, not for modeling the larger environmental variance that would appear in a multi-host deployment study.

The second-batch follow-up results support the same interpretation. In the external validation dataset, R1 is a real-project-derived `fontspec` article example with native font selection

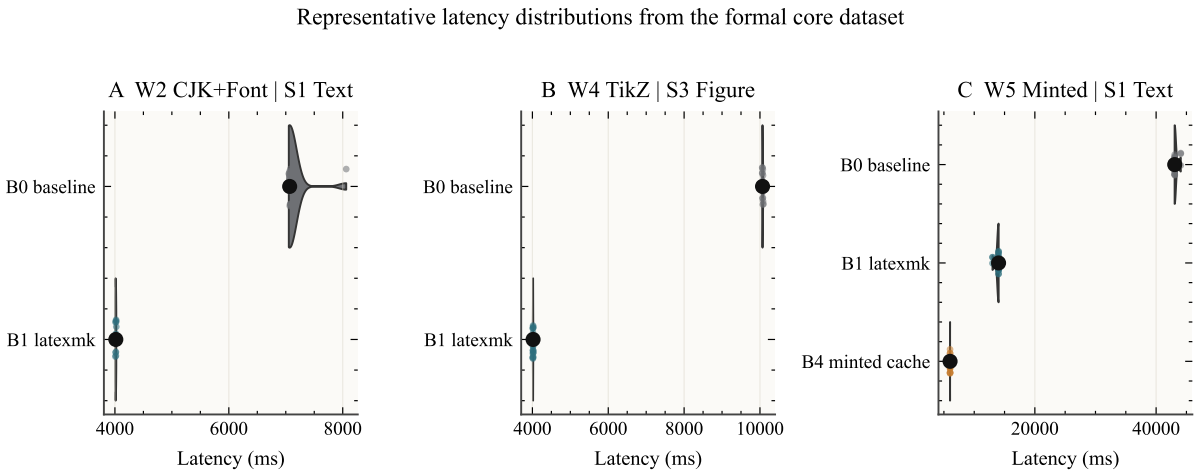

Figure 2: Representative per-run latency distributions. Panel A shows `W2` text edits, panel B shows `W4` figure edits, and panel C shows `W5` text edits with both `latexmk` and `minted` cache.

Table 6: Practical guidance distilled from the present artifact.

Case	Recommendation	Notes
Ordinary incremental writing	`B1 latexmk`	Best default for steady S1/S3/S4 loops on W1, W2, and W4.
Code-heavy minted documents	`B4 minted cache`	Strongest safe path in the current matrix: about 7.16x and roughly 37 seconds saved on incremental edits.
Clean-build or bib-sensitive loops	`Measure B0 and B1 locally`	`latexmk` is not guaranteed to win on S0 or S2.
Fidelity-risk acceleration paths	`Do not enable by default`	`B3` and `B6` remain diagnostic paths rather than default recommendations.

and an active bibliography path, while R2 is a real-project-derived PGFPlots example with figure regeneration pressure. These workloads are not synthetic variants of `W1`–`W5`; they are deliberately chosen outside the core matrix to test whether the `latexmk` pattern survives on document sources that were not originally authored for the benchmark. It does: clean builds and bibliography edits are slower than the direct XeLaTeX baseline at about 0.78x baseline speed, while incremental edits still improve by about 1.76x, again with full visual equality. In the B7 storage dataset, the passive Windows storage comparison between `d_repo_ntfs` and `c_profile_ntfs` preserves visual equality in all nine groups, but the `c_profile_ntfs` root is slower overall, with a median latency ratio of about 1.20x relative to the repository-local `D:` root. That observation is intentionally scoped: it may reflect profile-root background activity, Defender behavior, or metadata differences, but it is not presented here as a universal claim that one drive letter is always better than another. Appendix Tables 8 and 10 summarize these follow-up datasets directly.

5 Threats to Validity

The current artifact still runs on a single Windows host, so the reported latency distributions inevitably include host-specific effects such as storage layout, antivirus state, and local font availability. The workload mix also remains partly synthetic even after the two real-project-derived validation workloads were added, which means the benchmark is stronger as a controlled comparative study than as a claim about every class of TeX project. The storage follow-up is

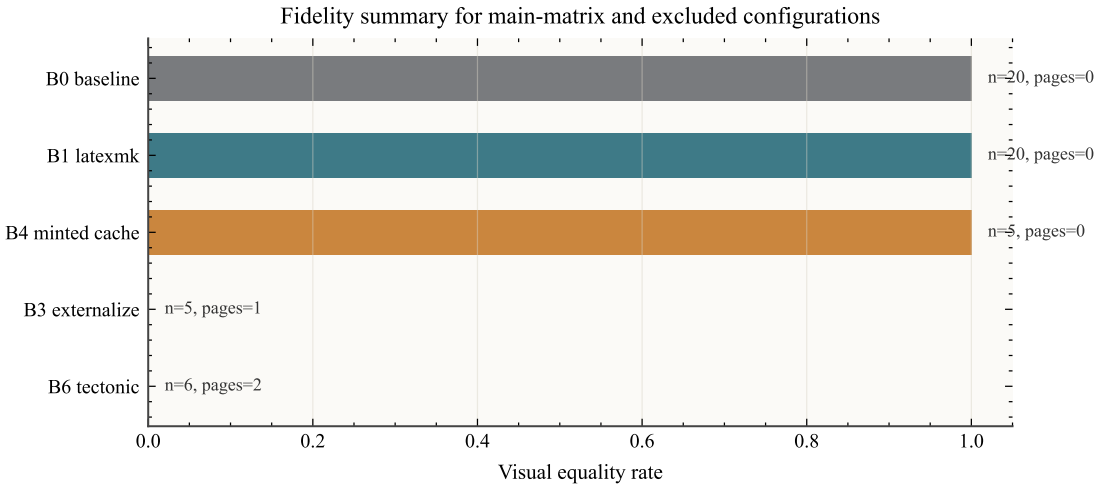

Figure 3: Visual-equality summary for the main-matrix configurations and the excluded `B3`/`B6` diagnostic paths. The core matrix remains fully visually equal, while triage reproduces persistent fidelity failures for the excluded configurations.

especially scoped: Dev Drive is intended as a dedicated Windows developer volume [12], but the current environment does not permit a full privileged Dev Drive provisioning or direct performance-mode manipulation. The benchmark therefore reports a Windows-local storage-root comparison under the observed system state, not a universal claim about all Dev Drive configurations. Finally, the current paper emphasizes descriptive results and reproducible artifacts; the appendix-level Wilcoxon analysis is informative, but it depends on recorded-order pairing rather than an explicit synchronized cross-config run identifier.

6 Conclusion

The current evidence supports a conservative recommendation set. For steady incremental writing loops, `latexmk` is the default win, but its gains are workload- and scenario-dependent rather than universal. For code-heavy documents, `minted` cache is the strongest safe optimization in the present matrix. Clean builds and bibliography edits should not be assumed to benefit from automation by default, and configurations with repeated visual diffs should remain diagnostic paths rather than default recommendations. Table 6 condenses that decision logic into a practical takeaway rather than leaving it scattered across the text. Fidelity should remain a release gate, not an afterthought, because configuration choices that look attractive from latency alone can still produce visible output differences. This leaves FastXeBench in a useful position for an arXiv-first release: the benchmark results are stable, the excluded configurations are explicitly scoped, and the repository now supports both reruns and paper regeneration from the recorded datasets.

7 Artifact Availability

The benchmark workloads, orchestration scripts, paper sources, generated tables and figures, and compact analysis artifacts are publicly available in the FastXeBench GitHub repository. The repository exposes a minimal rerun path through `build.ps1`, a paper-regeneration path through the scripts under `paper/scripts/`, and an isolated submission dry run for arXiv validation. Compact datasets and analysis CSVs are stored in the repository, while larger raw datasets may be distributed as release assets or archive attachments rather than committed directly into the main Git history.

A Statistical Details

B External Validation and Follow-up Studies

C Excluded Configurations

`B2` remains a compatibility-limited XeLaTeX negative result tied to the `mylatexformat` path [5]. `B3` delivered speedups on TikZ-heavy cases but produced persistent visual differences, so it is retained as an appendix-only negative result rather than a recommendation. `B6` remains development-only: Tectonic provides a useful cached-engine comparison point [15], but its output differences were stable enough in triage to keep it out of the main matrix. `B7` should be read as a scoped Windows storage-root comparison rather than a complete Dev Drive experiment, because the current environment can observe Microsoft Defender and volume metadata but cannot exercise the full privileged Dev Drive path [12].

References

[1] Moritz Beller, Christian Eilers, Fabian Tigges, and Gernot Ernst. Toward an empirical theory of feedback-driven development. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, 2018.

[2] CTAN Team. fontspec – advanced font selection in XeLaTeX and LuaLaTeX. `https://ctan.org/pkg/fontspec`, 2026. Accessed 2026-03-21.

[3] CTAN Team. latexmk – fully automated LaTeX document generation. `https://ctan.org/pkg/latexmk`, 2026. Accessed 2026-03-21.

[4] CTAN Team. minted – highlighted source code for LaTeX. `https://ctan.org/pkg/minted`, 2026. Accessed 2026-03-21.

[5] CTAN Team. mylatexformat – precompile a LaTeX document into a format. `https://ctan.org/pkg/mylatexformat`, 2026. Accessed 2026-03-21.

[6] CTAN Team. pgf – create PostScript and PDF graphics in TeX. `https://ctan.org/pkg/pgf`, 2026. Accessed 2026-03-21.

[7] Sebastian Erdweg, Tijs van der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R. Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. A sound and optimal incremental build system with dynamic dependencies. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 2015.

[8] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. The SPACE of developer productivity. *Communications of the ACM*, 64(12):42–50, 2021.

[9] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications*, 2007.

[10] Ciera Jaspan and Caitlin Green. Developer productivity for humans, part 4: Build latency, predictability, and developer productivity. *IEEE Software*, 40(4):122–129, 2023.

[11] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management*, 2013.

[12] Microsoft. Set up a dev drive on Windows 11. https://learn.microsoft.com/en-us/windows/dev-drive/, 2026. Microsoft Learn, accessed 2026-03-21.

[13] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte: Theory and practice. *Journal of Functional Programming*, 30, 2020.

[14] Jovyn Tan and Manuel Rigger. Inconsistencies in tex-produced documents. *arXiv preprint arXiv:2407.15511*, 2024.

[15] Tectonic Project. Building documents with the Tectonic command-line tool. https://tectonic-typesetting.github.io/book/latest/v2cli/build.html, 2026. Accessed 2026-03-21.

Table 7: Appendix statistics for the non-baseline main-matrix configurations. IQR is reported as $[Q_1, Q_3]$, CI is the 95% bootstrap interval for the group median, and Holm-adjusted p values come from two-sided Wilcoxon signed-rank tests that pair runs by within-group recorded order after sorting by `run_id`.

Workload	Scenario	Config	Median (s)	IQR (s)	95% CI (s)	Speedup	Holm p
W1 Text+Math	S0 clean	B1 latexmk	8.02	[8.01, 8.02]	[8.01, 8.02]	0.88	<0.001
W1 Text+Math	S1 text	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W1 Text+Math	S2 bib	B1 latexmk	8.02	[8.01, 8.02]	[8.01, 8.02]	0.88	<0.001
W1 Text+Math	S3 figure	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W1 Text+Math	S4 preamble	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S0 clean	B1 latexmk	9.02	[9.01, 9.02]	[9.01, 9.02]	0.78	<0.001
W2 CJK+Font	S1 text	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S2 bib	B1 latexmk	9.02	[9.01, 9.02]	[9.01, 9.02]	0.78	<0.001
W2 CJK+Font	S3 figure	B1 latexmk	4.01	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S4 preamble	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W4 TikZ	S0 clean	B1 latexmk	10.02	[10.01, 10.02]	[10.01, 10.02]	1.01	<0.001
W4 TikZ	S1 text	B1 latexmk	4.02	[4.02, 4.02]	[4.02, 4.02]	2.50	<0.001
W4 TikZ	S2 bib	B1 latexmk	10.02	[10.01, 10.02]	[10.02, 10.02]	1.01	<0.001
W4 TikZ	S3 figure	B1 latexmk	4.02	[4.01, 4.02]	[4.02, 4.02]	2.51	<0.001
W4 TikZ	S4 preamble	B1 latexmk	4.01	[4.01, 4.02]	[4.01, 4.02]	2.51	<0.001
W5 Minted	S0 clean	B1 latexmk	38.02	[38.01, 38.02]	[38.01, 38.02]	1.13	<0.001
W5 Minted	S0 clean	B4 minted cache	19.02	[19.01, 19.02]	[19.01, 19.02]	2.26	<0.001
W5 Minted	S1 text	B1 latexmk	14.02	[14.01, 14.02]	[14.01, 14.02]	3.07	<0.001
W5 Minted	S1 text	B4 minted cache	6.02	[6.01, 6.02]	[6.01, 6.02]	7.16	<0.001
W5 Minted	S2 bib	B1 latexmk	38.01	[38.01, 38.02]	[38.01, 38.02]	1.13	<0.001
W5 Minted	S2 bib	B4 minted cache	16.02	[16.01, 16.02]	[16.01, 16.02]	2.69	<0.001
W5 Minted	S3 figure	B1 latexmk	14.01	[13.76, 14.01]	[14.01, 14.01]	3.07	<0.001
W5 Minted	S3 figure	B4 minted cache	6.01	[6.01, 6.02]	[6.01, 6.01]	7.16	<0.001
W5 Minted	S4 preamble	B1 latexmk	14.01	[14.01, 14.02]	[14.01, 14.02]	3.07	<0.001
W5 Minted	S4 preamble	B4 minted cache	6.01	[6.01, 6.02]	[6.01, 6.02]	7.16	<0.001

Table 8: External validation rollup for the two real-project-derived workloads.

Workload	Clean ×	Bib ×	Incremental ×	Visual eq.
R1 fontspec	0.78	0.78	1.76	1.00
R2 pgfplots	0.79	0.78	1.76	1.00

Table 9: Anatomy of the two real-project-derived external-validation workloads.

Workload	Pages	Lines	Bib	`fontspec`	Bib-active	PGFPlots	Why this workload matters
R1 fontspec	2	79	2	yes	yes	no	Native-font article example outside the synthetic core matrix.
R2 pgfplots	2	96	2	no	yes	yes	Plot-heavy figure example outside the synthetic core matrix.

Table 10: Scoped Windows storage follow-up. The baseline mode is the repository-local `D:` root.

Mode	Groups	Volume	Median ratio	Visual eq.
d_repo_ntfs	9	`D:/NTFS`	1.00	1.00
c_profile_ntfs	9	`C:/NTFS`	1.20	1.00